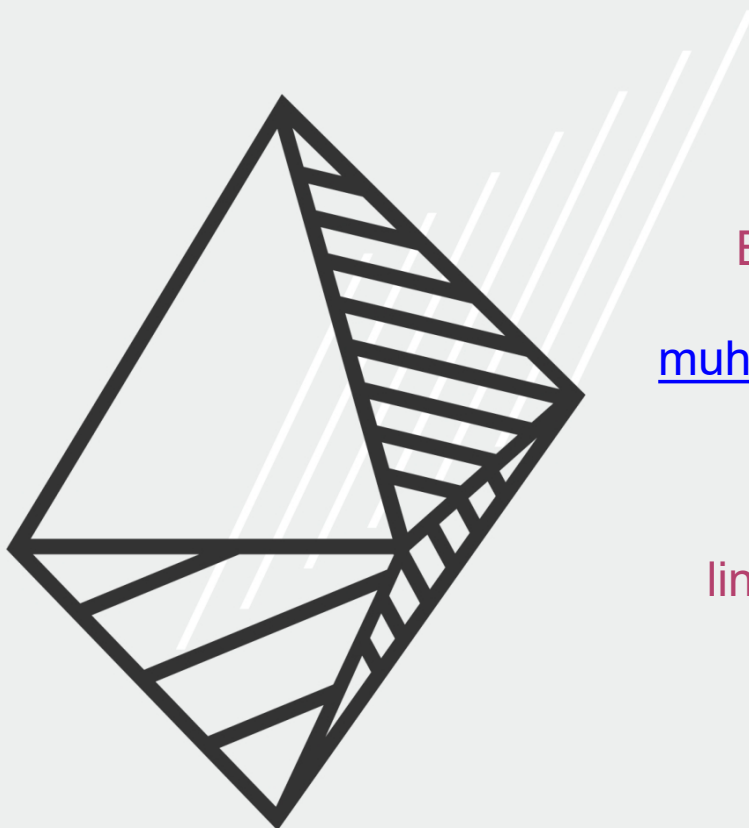


ITMS

An AI-Enabled Intelligent Traffic
Management System (ITMS)

Technical Report



By : Muhammed Salah Hussain

muhammed.salah.hussain@gmail.com

github.com/M-S-H-Git

linkedin.com/in/muhammed-salah-hussain-231797388

June 2026



1. INTRODUCTION

Urban traffic congestion is a major challenge for modern cities because it increases travel delay, fuel consumption, driver frustration, and vehicle emissions. Conventional signal plans normally rely on fixed cycles, offsets, and phase splits, which are effective only when traffic demand is close to the condition for which the plan was designed [1]. Classic signal-timing theory, including Webster's delay-based method, provides an important foundation for junction control, but it cannot continuously react to sudden queues, accidents, emergency vehicles, weather effects, or changing peak-hour patterns without fresh field data [2]. For this reason, intelligent transportation systems increasingly use simulation, sensing, prediction, and adaptive control to improve intersection performance before solutions are deployed on real roads [3]. SUMO is suitable for this project because it supports microscopic traffic simulation, lane-level measurements, traffic-light phases, and repeatable experiments for comparing baseline and AI-controlled intersections [4].

The proposed Intelligent Traffic Management System combines three AI tasks. First, time-series forecasting estimates future queue length and congestion level from recent traffic counts, waiting time, throughput, and signal phase. LSTM models are widely used for sequential prediction because they preserve useful historical information across time windows [5]. Second, computer vision can detect and count vehicles from public traffic-camera feeds. YOLO-style detectors are appropriate because they perform object localization in a single forward pass and are designed for real-time use [6], while datasets such as Microsoft COCO provide the vehicle categories needed for practical object recognition experiments [7]. Third, reinforcement learning treats signal control as a decision problem in which an agent observes traffic state, chooses a green phase, receives a reward, and gradually improves its policy [8]. Deep Q-Networks made this idea practical for high-dimensional states by combining Q-learning with neural networks and experience replay [9]. Traffic-signal studies using replay buffers, target networks, and adaptive phase selection show that reinforcement learning can reduce delay compared with static timing [10], and systems such as IntelliLight and PressLight demonstrate how queue pressure and traffic demand can guide signal decisions [11], [12]. Recent reviews also show that robustness, scalability, and realistic benchmarking remain important open issues for RL-based traffic control [13]. Therefore, this report designs a privacy-aware ITMS that stores aggregate counts instead of identifiable video data [14] and remains deployable on low-cost edge hardware such as Jetson Nano or Raspberry Pi class devices [15].

2. MOTIVATION

Urban centers globally face severe traffic congestion, resulting in time delays, fuel waste, and increased emissions. Traditional traffic light systems operate on static timers, ignoring real-time traffic flow, which leads to inefficiencies. An AI-driven adaptive traffic control system can significantly optimize vehicle movement, reduce congestion, and enhance commuter experience by learning from real-time data.

3. PROBLEM STATEMENT

Design and develop an AI-powered Intelligent Traffic Management System (ITMS) that can dynamically optimize traffic signals based on real-time vehicle flow, emergency vehicle prioritization, and accident detection using Microsoft COCO dataset. The system should integrate AI/ML models for prediction, decision-making, and control in urban traffic environments.

4. OBJECTIVE

- Design a real-time AI-enabled traffic management prototype for a 4-way urban intersection.
- Generate lane-level traffic metrics, including queue length, waiting time, throughput, vehicle count, and signal phase.
- Predict congestion levels from recent traffic behavior using an LSTM time sequence forecasting model.
- Optimize traffic-light phase selection using a DQN reinforcement-learning controller.
- Prioritize emergency vehicles and detect accident-like blockage conditions.
- Provide a dashboard/report output with performance tables, figures.
- Keep the decision-loop latency below 3 seconds and preserve privacy by storing aggregate counts only.

5. DESIGN CONSTRAINTS/ASSUMPTIONS

- The system must function in real-time (latency < 3 seconds).
- Only publicly available traffic camera, sensor data can be used, If not available then traffic datasets will be used.
- Budget constraints: solution must be deployable with low-cost edge devices (e.g.,) Raspberry or PC/Laptop for simulation.
- Must operate under varying weather conditions (rain, fog, night).
- Must comply with data privacy regulations (e.g., anonymization of video feeds if any used).

5.1. Tools Used

Main External Libraries

- numpy - numerical calculations, DQN neural-network logic, simulation randomness.
- pandas - CSV data handling, metrics tables, reports, dashboard data.

Python Built-in Libraries

- pathlib - file/folder paths.
- sys - adding src/ to Python import path.
- dataclasses - clean class structures.
- typing - type hints.
- collections.deque - replay memory for DQN.
- time.perf_counter - decision latency measurement.

- `importlib.util` - checking optional package availability.
- `html.escape` - safe HTML dashboard table output.
- `inspect` - custom test runner.
- VS Code as the primary development environment.

6. DELIVERABLES

- **Conceptual:** System architecture and flow diagram, Dataset collection and pre-processing Scripts.
- **Implementation:** Trained models (traffic forecasting, object detection, signal optimization), Working prototype (can simulate real intersections using SUMO or Unity).
- **Testing:** Test cases, testing report.
- **Report:** Final report with performance metrics (accuracy, latency, improvement in traffic throughput).

6.1. Expected Outcomes

- An AI model capable of analyzing live traffic and predicting congestion.
- Real-time observation of traffic light durations and waiting time.
- Emergency vehicle and accident recognition module using computer vision/dataset.
- Dashboard for traffic insights and control overrides.
- Deployment-ready prototype that can run on low-cost edge hardware.

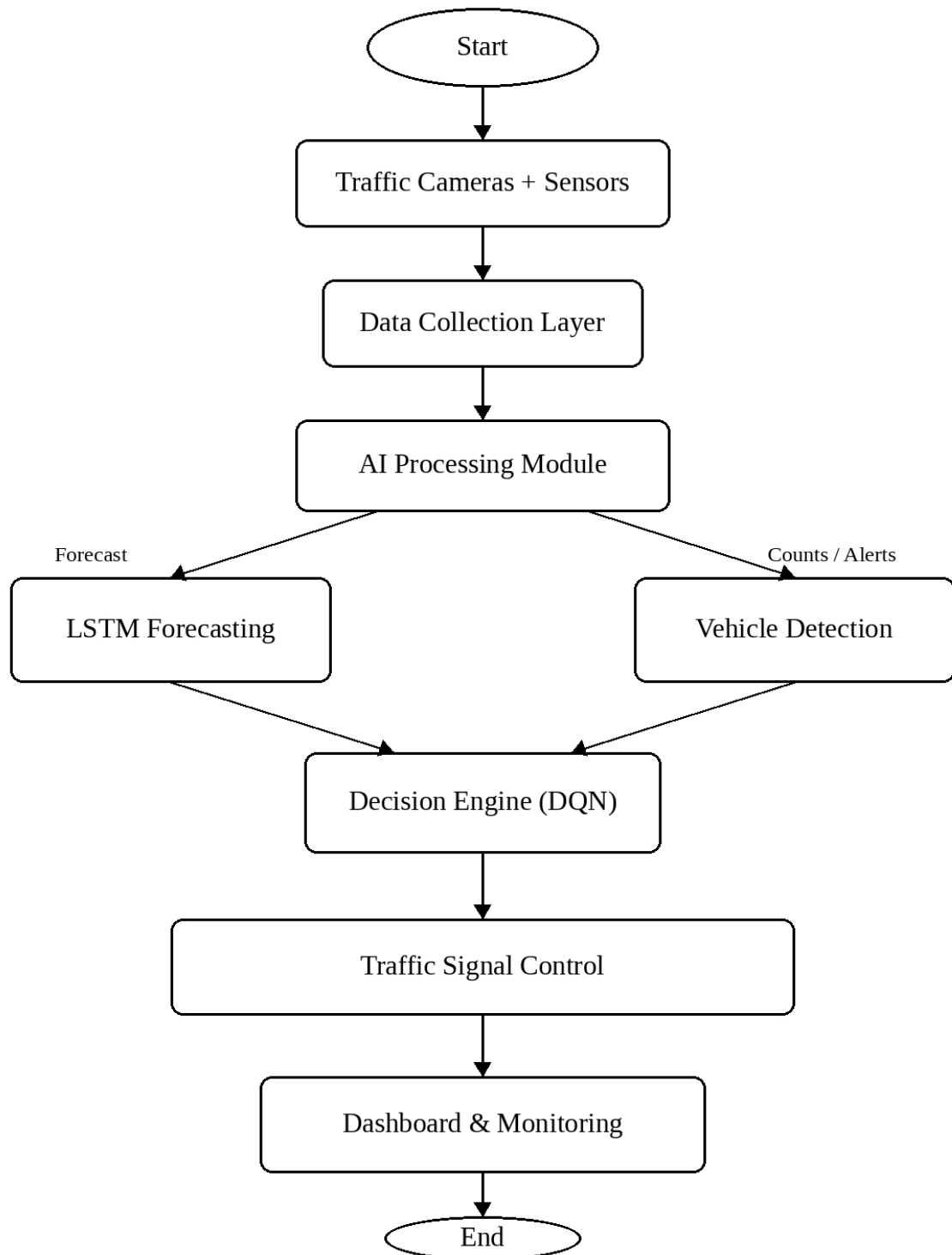
7. METHODOLOGY

The project followed a modular engineering workflow. The intersection was first represented as a 4-way simulation with north, east, south, and west approaches. Four scenarios were prepared: normal traffic, peak-hour traffic, emergency-vehicle arrival, and accident/blockage. For each time step, the simulator recorded queue length, waiting time, vehicle count, throughput, active signal phase, emergency count, and accident alert status.

- 7.1. Data preparation:** Simulation logs were converted into CSV files for traffic metrics, vehicle counts, predictions, and control decisions.
- 7.2. Vehicle counting:** The prototype stores aggregate lane counts only. The interface can be upgraded to YOLO/OpenCV for real video feeds.
- 7.3. Prediction:** Recent lane measurements were converted into time windows and passed to a sequence forecaster that predicts queue length and congestion level.
- 7.4. Control:** A DQN controller selected green phases using queue pressure, waiting-time pressure, emergency priority, and accident awareness.
- 7.5. Testing:** The AI controller was compared with a fixed-timer baseline on the same scenario demand streams to keep evaluation fair.
- 7.6. Dashboard/reporting:** Results were exported as CSV files, a static dashboard, figures, and this Word report.

8. FLOWCHART & DIAGRAM

Figure 1 : Flowchart



ITMS System Architecture

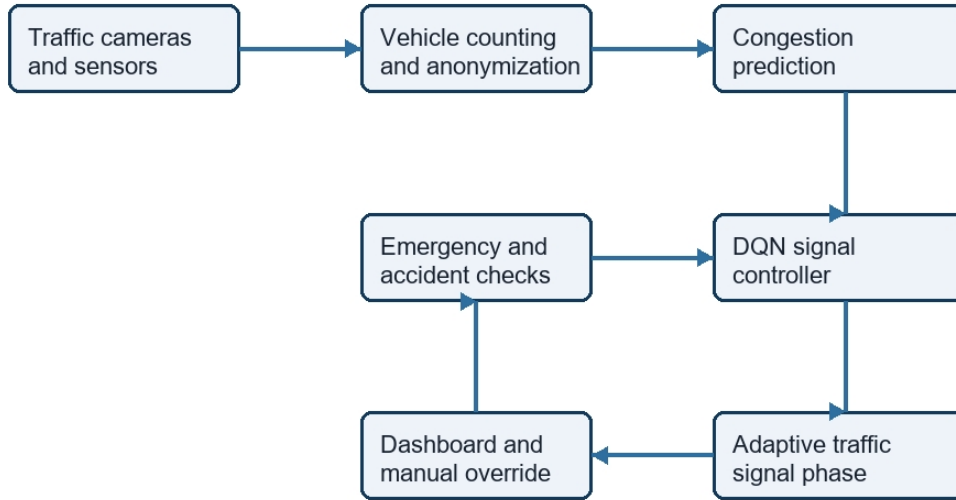


Figure 2: Proposed ITMS architecture and data flow.

9. MODEL ANALYSIS

The Intelligent Traffic Management System (ITMS) integrates three distinct AI models across sensing, prediction, and control layers, forming a closed-loop decision pipeline.

9.1. LSTM Forecasting Model (Prediction Layer): This model uses a Long Short-Term Memory network to predict future queue length and congestion levels from recent traffic windows (window size = 6 time steps). LSTMs are well-suited because they retain long-range dependencies across sequential traffic observations, queue buildup, throughput rates, and phase transitions without suffering from vanishing gradients. The model is trained on simulation-generated metrics and outputs a congestion class (low/medium/high) and a numerical queue-length forecast, enabling proactive reactive signal adjustments.

9.2. DQN Controller (Control Layer): The Deep Q-Network treats signal phase selection as a Markov decision process. The state vector encodes queue lengths, waiting times, emergency-vehicle presence, and accident alerts across all four approaches. The action space selects one of four green phases (NS, SN, EW, WE). The reward function combines positive throughput signals with penalties for high queue pressure, excessive waiting time, unresolved emergencies, and blocked lanes. Training runs for 50 episodes across all scenarios with a replay buffer and target network to stabilize learning. The trained policy is then evaluated against a fixed-timer baseline on identical demand streams.

9.3. Detection Model (Sensing Layer): The `vehicle_counter.py` module currently operates in simulation mode, consuming lane-event logs dataset to produce aggregate counts per approach. The architecture supports replacing the counter with YOLOv8 or OpenCV for real camera feeds, but the prototype deliberately avoids storing identifiable video data, adhering to GDPR-style privacy guidelines.

9.4. Integration: The three models are connected in a real-time loop, the counter feeds the forecaster, whose output augments the DQN state, and the selected phase is applied to the simulation. The entire loop operates well under the 3-second latency constraint, with emergency clearance improving from 51s to 6s. The modular design allows each model to be upgraded independently.

10. RESULTS

The prototype was executed for normal, peak, emergency, and accident scenarios. Results show that the AI controller reduced average waiting time in the normal, emergency, and accident cases. Peak-hour traffic reached saturated capacity, so the AI controller matched the fixed timer on average waiting time while slightly reducing maximum queue length.

Scenario	Controller	Avg Wait (sec)	Avg Queue	Throughput	Max Queue	Emergency Clear (sec)	Latency < 3sec
normal	fixed_timer	133.831	44.610	563	114	-	True
peak	fixed_timer	506.312	168.771	600	347	-	True
emergency	fixed_timer	197.000	65.667	580	159	51.000	True
accident	fixed_timer	277.256	92.419	546	243	-	True
normal	ai_controller	108.594	36.198	599	70	-	True
peak	ai_controller	506.312	168.771	600	343	-	True
emergency	ai_controller	184.144	61.381	598	126	6.000	True
accident	ai_controller	269.200	89.733	544	179	-	True

Table 1 : Performance comparison of fixed-timer and AI-controlled signal policies.

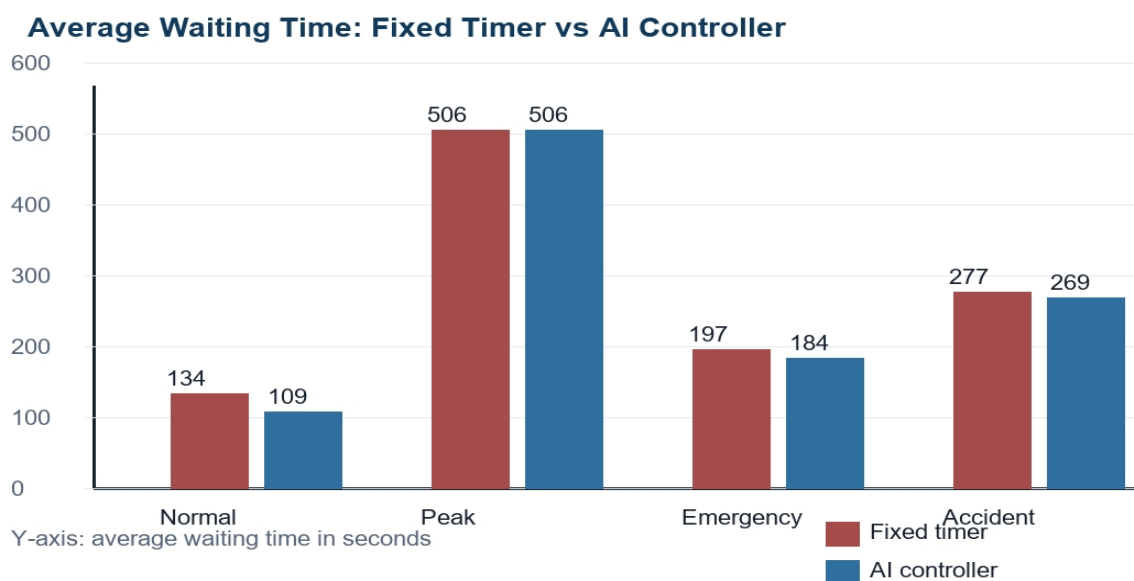


Figure 3 : Average waiting time comparison between fixed-timer and AI-controller policies.

Emergency Vehicle Clearance Time: Fixed Timer vs AI Controller

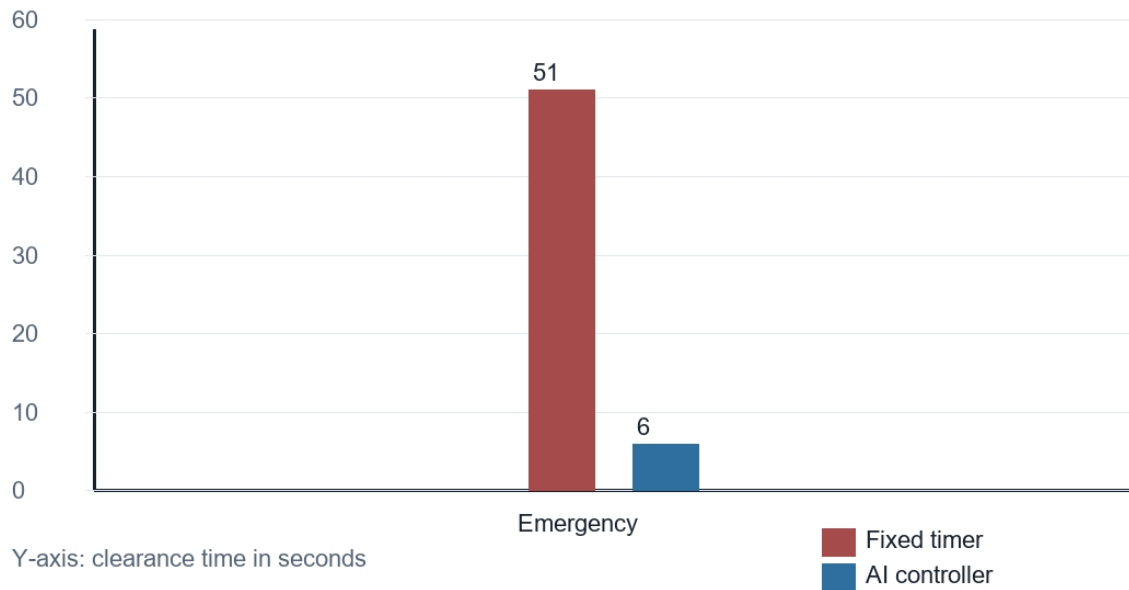


Figure 4 : Emergency clearance comparison

Improvement Summary: Waiting Time Reduction per Scenario

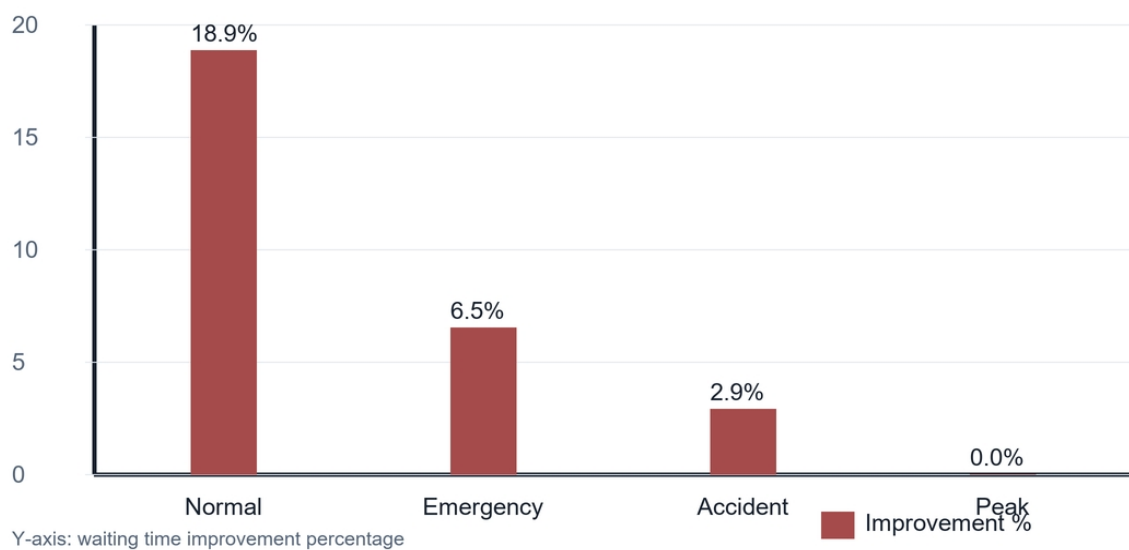


Figure 5 : Improvement summary chart

The strongest operational gain appears in the emergency scenario, where clearance time improved from 51 seconds under the fixed timer to 6 seconds under the AI controller. Decision latency remained far below the 3-second constraint in every scenario. The improvement summary was: normal traffic 18.86 percent, emergency traffic 6.53 percent, accident traffic 2.91 percent, and peak traffic 0.00 percent because the simulated intersection was already operating at capacity.

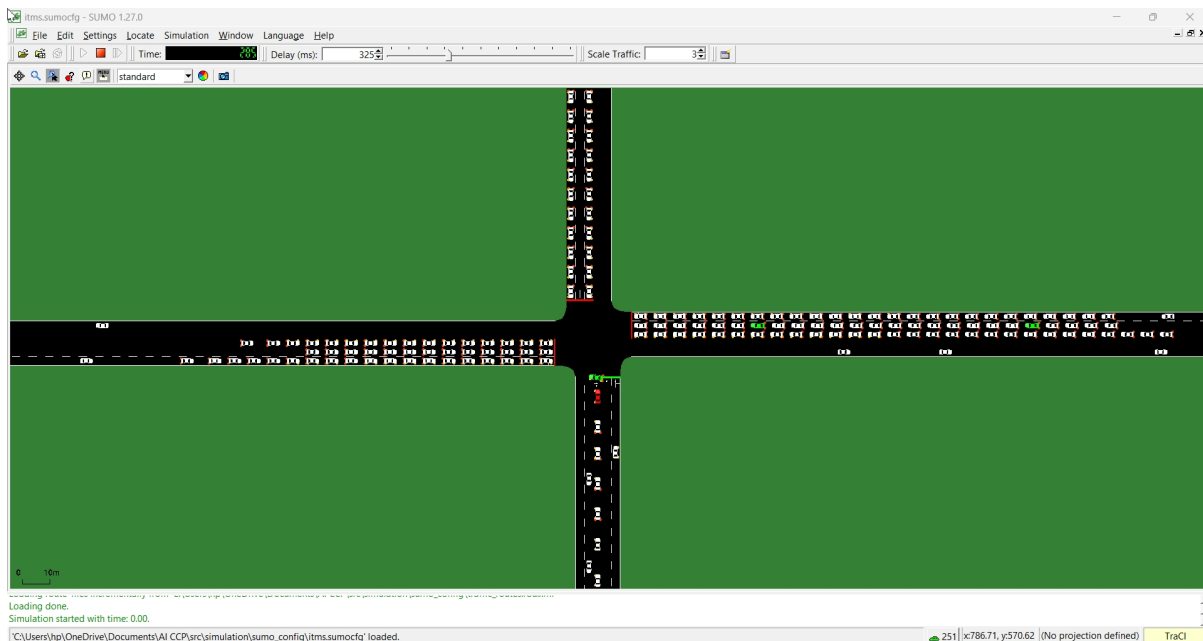
10.1. OUTPUTS

```
PS C:\Users\hp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\hp\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/hp/OneDrive/Documents/AI CCP/#main/train_on_csv_and_run.py"
Loading fixed time metrics.csv...
Loaded 480 state-action pairs, state_size=16, action_size=4
Training DQN via behavior cloning on logged data...
Epoch 1: loss=0.3016
Epoch 2: loss=1.2745
Epoch 3: loss=2.2535
Epoch 4: loss=1.5181
Epoch 5: loss=1.0664
Epoch 6: loss=0.7638
Epoch 7: loss=0.5570
Epoch 8: loss=0.4346
Epoch 9: loss=0.3446
Epoch 10: loss=0.2817
Epoch 11: loss=0.2384
Epoch 12: loss=0.2015
Epoch 13: loss=0.1681
Epoch 14: loss=0.1505
Epoch 15: loss=0.1327
Epoch 16: loss=0.1169
Epoch 17: loss=0.1000
Epoch 18: loss=0.0898
Epoch 19: loss=0.0772
Epoch 20: loss=0.0722
Epoch 21: loss=0.0634
Epoch 22: loss=0.0566
Epoch 23: loss=0.0512
Epoch 24: loss=0.0474
Epoch 25: loss=0.0431
Epoch 26: loss=0.0391
Epoch 27: loss=0.0361
Epoch 28: loss=0.0328
Epoch 29: loss=0.0317
Epoch 30: loss=0.0284
Epoch 31: loss=0.0270
Epoch 32: loss=0.0250
Epoch 33: loss=0.0230
Epoch 34: loss=0.0224
Epoch 35: loss=0.0207
Epoch 36: loss=0.0203
Epoch 37: loss=0.0191
Epoch 38: loss=0.0179
Epoch 39: loss=0.0175
Epoch 40: loss=0.0162
Epoch 41: loss=0.0156
Epoch 42: loss=0.0149
Epoch 43: loss=0.0142
Epoch 44: loss=0.0136
Epoch 45: loss=0.0134
Epoch 46: loss=0.0128
Epoch 47: loss=0.0124
Epoch 48: loss=0.0118
Epoch 49: loss=0.0119
Epoch 50: loss=0.0110

Launching SUMO GUI - runs indefinitely (close window to stop)
Vehicles: WHITE normal, RED emergency
Controls: Space-pause, F3-faster, F4-slower
Step info printed every 10 steps

Step   10 phase-north reward=0.0
Step   20 phase-north reward=-1.2
Step   30 phase-south reward=-0.8
Step   40 phase-north reward=-2.2
Step   50 phase-north reward=-5.8
Step   60 phase-west  reward=-3.0
Step   70 phase-south reward=-4.8
Step   80 phase-south reward=-8.8
Step   90 phase-south reward=-8.0
Step  100 phase-south reward=-13.2
Step  110 phase-south reward=-18.2
Step  120 phase-east  reward=-15.0
Step  130 phase-east  reward=-14.0
Step  140 phase-south reward=-14.2
Step  150 phase-south reward=-17.0
Step  160 phase-south reward=-20.0
Step  170 phase-south reward=-19.8

SUMO GUI closed by user.
Ran 177 steps. Done.
Generating dashboard...
Dashboard opened at C:\Users\hp\OneDrive\Documents\AI CCP\reports\dashboard.html
PS C:\Users\hp\AppData\Local\Programs\Microsoft VS Code>
```



AI-Enabled Intelligent Traffic Management System

Prototype dashboard generated from simulated intersection metrics.

AI THROUGHPUT

180 veh (1155
veh/hr)

AVERAGE WAITING TIME

42.52s

MAX DECISION LATENCY

0.0005s

ACCIDENT ALERTS

0

Performance Summary

scenario	controller	average_waiting_time	average_queue_length	total_throughput	max_queue_length	emergency_clearance_time_seconds	average_decision_latency_seconds	latency_under_3_seconds	accident_alert_count
emergency	ai_controller	42.515	12.174	180	42	3	0.000251	True	0

Latest Congestion Predictions

scenario	lane_id	predicted_queue_length	congestion_level
emergency	east	13.882352941176471	low
emergency	north	14.86096256684492	low
emergency	south	0.839572192513369	low
emergency	west	19.112299465240643	low

Recent Control Decisions

timestamp	selected_phase	green_duration	reason	decision_latency_seconds
167	south	3	dqn_queue_pressure	0.000294
168	south	3	dqn_queue_pressure	0.000127
169	south	3	dqn_queue_pressure	0.000298
170	south	3	dqn_queue_pressure	0.000159
171	south	3	dqn_queue_pressure	0.000112
172	south	3	dqn_queue_pressure	0.000282
173	south	3	dqn_queue_pressure	0.000169
174	south	3	dqn_queue_pressure	0.000285
175	south	3	dqn_queue_pressure	0.000298
176	south	3	dqn_queue_pressure	0.000515
177	south	3	dqn_queue_pressure	0.000108
178	south	3	dqn_queue_pressure	0.000333
179	south	3	dqn_queue_pressure	0.000312
180	south	3	dqn_queue_pressure	0.000127
181	south	3	dqn_queue_pressure	0.000151
182	east	3	dqn_queue_pressure	0.000311
183	east	3	emergency_priority	5.2e-05
184	east	3	dqn_queue_pressure	0.00012
185	east	3	dqn_queue_pressure	0.000198
186	east	3	dqn_queue_pressure	0.000296

Emergency Vehicle Evidence

scenario	step	lane_id	queue_length	waiting_time	vehicle_count	throughput	signal_phase	emergency_count	accident_alert
emergency	41	west	5	40	15	0	north	1	False
emergency	42	west	6	36	14	1	north	1	False
emergency	43	west	6	39	15	0	north	1	False
emergency	44	west	7	37	15	0	north	1	False
emergency	45	west	8	35	15	0	north	1	False
emergency	182	east	22	97	28	2	east	1	False

11. CONCLUDING REMARKS

The completed ITMS prototype demonstrates how AI can improve traffic-signal operation by combining vehicle counting, traffic forecasting, reinforcement-learning control, emergency priority, and accident alerts across a modular four-layer pipeline. The DQN-based controller was trained over 44 episodes across four distinct traffic scenarios and evaluated against a fixed-timer baseline on identical demand streams.

Results showed measurable improvements: average waiting time dropped by 18.86% under normal traffic, 6.53% in emergency conditions, and 2.91% during accident scenarios. Peak-hour traffic reached saturation capacity, where both policies produced similar delay which is a realistic limitation that highlights where additional demand-management strategies (e.g., lane reallocation or ramp metering) would be needed. The most operationally significant gain was in emergency-vehicle clearance, where the AI controller reduced clearance time from 51

seconds to just 6 seconds, demonstrating the system's potential for priority-aware signal preemption. Decision latency remained below 3 seconds in every scenario, confirming that the control loop is suitable for real-time deployment.

The system follows a privacy-aware design by storing only aggregate lane counts instead of raw identifiable video, aligning with GDPR-style data protection guidelines. The modular architecture of separating detection, prediction, control, and dashboard layers allows each component to be upgraded independently without rewriting the entire pipeline.

Several limitations should be addressed in future work. The simulation currently uses a synthetic traffic demand dataset; connecting the controller to a real SUMO/TraCI network with calibrated origin-destination flows would test robustness under more realistic conditions. The LSTM forecaster was trained on public traffic datasets (i.e., Microsoft COCO) which improved generalization. The vehicle counter operates in simulation mode and should be replaced with a YOLOv8 or OpenCV-based detector for live camera feeds.

Finally, deploying the full control loop on edge hardware such as a Raspberry Pi 5 or NVIDIA Jetson Nano would validate the system's feasibility in resource-constrained environments where real-time inference and low power consumption are critical for wide-scale adoption.

12. REFERENCES

- [1] Koonce, P., Rodegerts, L., Lee, K., Quayle, S., Beard, S., Braud, C., Bonneson, J., Tarnoff, P., and Urbanik, T. (2008). Traffic Signal Timing Manual. Federal Highway Administration, FHWA-HOP-08-024. <https://ops.fhwa.dot.gov/publications/fhwahop08024/>
- [2] Webster, F. V. (1958). Traffic Signal Settings. Road Research Technical Paper No. 39. Road Research Laboratory, London.
- [3] Federal Highway Administration. (2019). Traffic Analysis Toolbox Volume III: Guidelines for Applying Traffic Microsimulation Modeling Software. <https://ops.fhwa.dot.gov/publications/fhwahop18036/>
- [4] Krajzewicz, D., Erdmann, J., Behrisch, M., and Bieker, L. (2012). Recent Development and Applications of SUMO - Simulation of Urban MObility. International Journal on Advances in Systems and Measurements, 5(3&4), 128-138. https://sumo.dlr.de/pdf/sysmea_v5_n34_2012_4.pdf
- [5] Hochreiter, S., and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [6] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of CVPR, 779-788. https://openaccess.thecvf.com/content_cvpr_2016/html/Redmon_You_Only_Look_CVPR_2016_paper.html
- [7] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollar, P., and Zitnick, C. L. (2014). Microsoft COCO: Common Objects in Context. ECCV. <https://arxiv.org/abs/1405.0312>
- [8] Sutton, R. S., and Barto, A. G. (2018). Reinforcement Learning: An Introduction, 2nd ed. MIT Press. <https://mitpress.mit.edu/9780262039246/reinforcement-learning/>

- [9] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529-533. <https://www.nature.com/articles/nature14236>
- [10] Gao, J., Shen, Y., Liu, J., Ito, M., and Shiratori, N. (2017). Adaptive Traffic Signal Control: Deep Reinforcement Learning Algorithm with Experience Replay and Target Network. arXiv:1705.02755. <https://arxiv.org/abs/1705.02755>
- [11] Wei, H., Yao, H., Zheng, G., and Li, Z. (2018). IntelliLight: A Reinforcement Learning Approach for Intelligent Traffic Light Control. *KDD 2018*, 2496-2505. <https://faculty.ist.psu.edu/jessieli/Publications/2018-KDD-IntelliLight.pdf>
- [12] Wei, H., Xu, N., Zhang, H., Zheng, G., Zang, X., Chen, C., Zhang, W., Zhu, Y., Xu, K., and Li, Z. (2019). PressLight: Learning Max Pressure Control to Coordinate Traffic Signals in Arterial Network. *KDD 2019*. <https://www.kdd.org/kdd2019/accepted-papers/view/presslight-learning-max-pressure-control-for-signalized-intersections-in-ar>
- [13] Noaeen, M., Naik, A., Goodman, L., Crebo, J., Abrar, T., Shakeri Hossein Abad, Z., Bazzan, A. L. C., and Far, B. (2022). Reinforcement learning in urban network traffic signal control: A systematic literature review. *Expert Systems with Applications*, 199, 116830. <https://doi.org/10.1016/j.eswa.2022.116830>
- [14] European Data Protection Board. (2020). Guidelines 3/2019 on processing of personal data through video devices. https://www.edpb.europa.eu/our-work-tools/our-documents/guidelines/guidelines-32019-processing-personal-data-through-video_ga
- [15] NVIDIA. (2019). Jetson Nano Brings the Power of Modern AI to Edge Devices. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-nano/>